

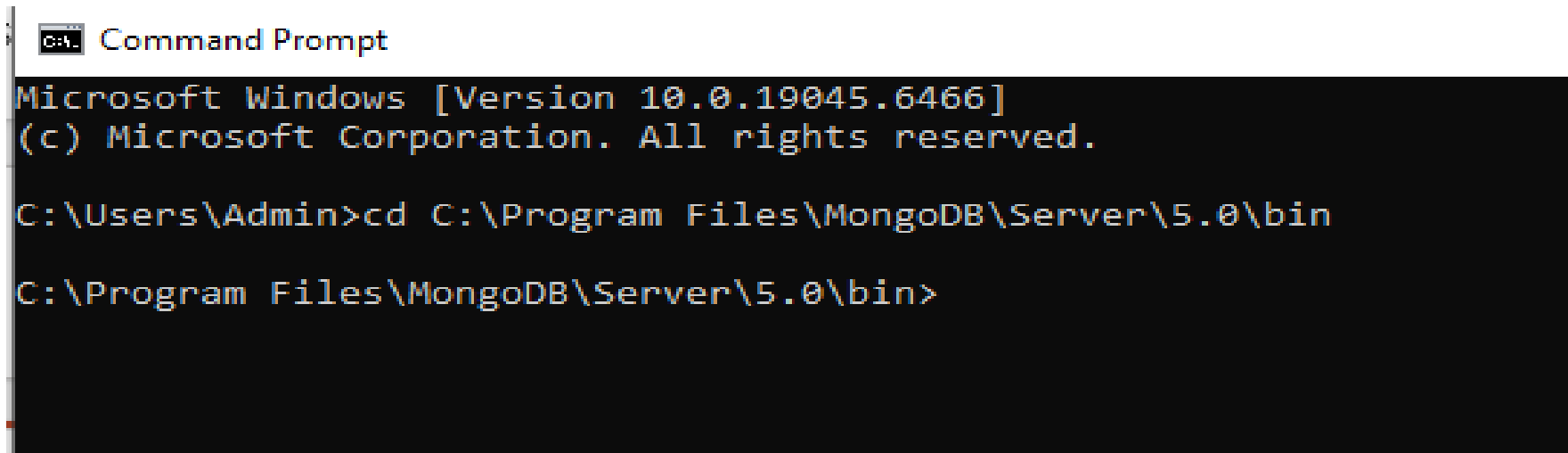
LAB PROGRAM 1

- **Using MongoDB, create a collection called transactions in database usermanaged (drop if it already exists) and bulk load the data from a json file, transactions.json. Upsert the record from the new file called transactions_upsert.json in Mongoddb.**

The task is:

- Create a database called **usermanaged**
- Create a collection called **transactions**
- Load data from a JSON file (**transactions.json**) into the collection
- Update or insert records from another file (**transactions_upsert.json**) using **upsert**

- Open command prompt: **cmd**
- Change the directory to **mongodb bin** (go to that folder copy the path)
- In command prompt type
- **cd** C:\Program Files\MongoDB\Server\5.0\bin
- Once the path is changed type **mongod**



```
C:\> Command Prompt
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Admin>cd C:\Program Files\MongoDB\Server\5.0\bin

C:\Program Files\MongoDB\Server\5.0\bin>
```

- **Open another command prompt** (to run the client/shell)
- Similarly copy bin path
- **C:\Users\DELL> cd C:\ProgramFiles\MongoDB\Server\8.0\bin**
- After that type **mongosh**
- **C:\Program Files\MongoDB\Server\8.0\bin>mongosh**

- Inside mongosh type the following

 Command Prompt - mongo

```
> use usermanaged
switched to db usermanaged
> db.createCollection("transactions")
{ "ok" : 1 }
> db.transactions.find()
>
```

- create two json file
- **transactions.json** and
- **transactions_upsert.json** containing set of documents.
- Save transactions.json (preferably in bin folder itself)

transactions.json

```
[
  {
    "_id": 1,
    "Name": "Somu",
    "Payment": { "Total": 600 },
    "Transaction": { "Price": 400 }
  },
  {
    "_id": 2,
    "Name": "Ravi",
    "Payment": { "Total": 500 },
    "Transaction": { "Price": 350 }
  },
  {
    "_id": 3,
    "Name": "Somu",
    "Payment": { "Total": 700 },
    "Transaction": { "Price": 450 }
  }
]
```

MongoDB Command Line Database Tools Download

The MongoDB Database Tools are a collection of command-line utilities for working with a MongoDB deployment. These tools release independently from the MongoDB Server schedule enabling you to receive more frequent updates and leverage new features as soon as they are available. See the [MongoDB Database Tools](#) documentation for more information.

Note: MongoDB Database Tools are distributed as open source (Apache 2.0), standalone products developed separately from the MongoDB Server.

Version	100.14.1	▼
Platform	Windows x86_64	▼
Package		

- **install mongoimport (Windows)**
- Download **MongoDB Database Tools** from MongoDB website (Search: *MongoDB Database Tools download*)
- Extract the ZIP file.
- Inside the extracted folder, open:
- Inside MongoDB folder create a folder named Tools
- Extract the zip file content (bin folder) to this folder
- C:\Program Files\MongoDB\Tools\bin

- Open the command prompt 3:- **cmd**

- **mongoimport - -db usermanaged - -collection transactions - -file transactions.json - -jsonArray**

- After that Type

- **“copy the path of mongoimport.exe file” - -db usermanaged - -collection transactions - -file “*copy paste the path of json file*” - -jsonArray**

- **"C:\Program Files\MongoDB\Tools\mongodb-database-tools-windows-x86_64-100.14.1\bin\mongoimport.exe" --db usermanaged --collection transactions --file "C:\Program Files\MongoDB\Server\5.0\bin\transactions.json" --jsonArray**

Output

After Insert command:-

```
C:\Program Files\MongoDB\Server\5.0\bin>"C:\Program Files\MongoDB\Tools\mongodb-database-tools-windows-x64\bin\mongo" --n transactions --file "C:\Program Files\MongoDB\Server\5.0\bin\transactions.json" --jsonArray
2026-02-18T10:28:18.183+0530    connected to: mongodb://localhost/
2026-02-18T10:28:18.719+0530    3 document(s) imported successfully. 0 document(s) failed to import.

C:\Program Files\MongoDB\Server\5.0\bin>
```

OUTPUT

```
usermanaged> db.transactions.find().sort({"_id":1})
[
  {
    _id: 1,
    Name: 'Somu',
    Payment: { Total: 600 },
    Transaction: { Price: 400 }
  },
  {
    _id: 2,
    Name: 'Ravi',
    Payment: { Total: 500 },
    Transaction: { Price: 350 }
  },
  {
    _id: 3,
    Name: 'Somu',
    Payment: { Total: 700 },
    Transaction: { Price: 450 }
  }
]
```

- Go to **Mongosh** prompt
- To confirm whether the data got loaded from json file
- Give **db.transactions.find().sort({"_id":1})**
- **Sorts the id in ascending order**

UPSERT

- create transactions_upsert.json file

```
[
  {
    "_id": 3,
    "Name": "Anusha",
    "Payment": { "Total": 500 },
    "Transaction": { "Price": 400 }
  },
  {
    "_id": 4,
    "Name": "Bhuvan",
    "Payment": { "Total": 1000 },
    "Transaction": { "Price": 800 }
  }
]
```

- upsert the data from transactions_upsert.json
- Back to command prompt 3, then type:-
- **mongoimport** - -db usermanaged - -collection transactions - -file "*copy paste the path of transactions_upsert.json file*" - jsonArray - -mode **upsert**
- "C:\Program Files\MongoDB\Tools\mongodb-database-tools-windows-x86_64-100.14.1\bin\mongoimport.exe" --db usermanaged --collection transactions --file "**C:\Users\Admin\Desktop\transactions_upsert.json**" --jsonArray --mode upsert

OUTPUT

After Upsert

```
2026-02-19T00:01:05.781+0530    connected to: mongodb://localhost/
2026-02-19T00:01:05.866+0530    2 document(s) imported successfully. 0 document(s) failed to import.
```

OUTPUT

- Go to mongosh
- Check whether the document is updated
- **db.transactions.find().sort({"_id":1})**

- **Document 3 is updated**
With new value

- **Document 4 is inserted**

```
usermanaged> db.transactions.find().sort({"_id":1})
[
  {
    _id: 1,
    Name: 'Somu',
    Payment: { Total: 600 },
    Transaction: { Price: 400 }
  },
  {
    _id: 2,
    Name: 'Ravi',
    Payment: { Total: 500 },
    Transaction: { Price: 350 }
  },
  {
    _id: 3,
    Name: 'Anusha',
    Payment: { Total: 500 },
    Transaction: { Price: 400 }
  },
  {
    _id: 4,
    Name: 'Bhuvan',
    Payment: { Total: 1000 },
    Transaction: { Price: 800 }
  }
]
```

- **Summary :-**

- Open cmd1 for mongod (starting the **MongoDB server process.**)
- You may get waiting for connection
- Open cmd2 for mongosh (all queries here)
- Open cmd3 for mongoimport tool
- Create transactions.json with all documents
- Load all the documents to collection
- Create transactions_upsert.json with all few modifications
- Check whether update is happening using upsert mode

Lab program 2

Query MongoDB with Conditions: [Create appropriate collection with necessary documents to answer the query]

- a. Find any record where Name is Somu
- b. Find any record where total payment amount (Payment.Total) is 600.
- c. Find any record where price (Transaction.price) is between 300 to 500.
- d. Calculate the total transaction amount by adding up Payment.Total in all records.

- Note: Using same collection named transaction used in lab program 1 for this lab program.
- **Else** create a database and collection
- Go to mongosh
- use usermanaged

Create collections as:-

```
db.transactions.insertMany([
  {
    _id: 1,
    Name: "Somu",
    Payment: { Total: 600 },
    Transaction: { Price: 400 }
  },
  {
    _id: 2,
    Name: "Ravi",
    Payment: { Total: 500 },
    Transaction: { Price: 350 }
  },
  {
    _id: 3,
    Name: "Anusha",
    Payment: { Total: 500 },
    Transaction: { Price: 400 }
  },
  {
    _id: 4,
    Name: "Bhuvan",
    Payment: { Total: 1000 },
    Transaction: { Price: 800 }
  }
])
```

- a. Find any record where Name is Somu

db.transactions.find({ "Name": "Somu" })

```
> db.transactions.find({"Name":"Somu"})
{ "_id" : 1, "Name" : "Somu", "Payment" : { "Total" : 600 }, "Transaction" : { "Price" : 400 } }
```

b. Find any record where total payment amount (Payment.Total) is 600.

db.transactions.find({ "Payment.Total": 600 })

```
usermanaged> db.transactions.find({"Payment.Total":600})
[
  {
    _id: 1,
    Name: 'Somu',
    Payment: { Total: 600 },
    Transaction: { Price: 400 }
  }
]
```

- Find any record where price (Transaction.price) is between 300 to 500.

db.transactions.find({ "Transaction.Price": { \$gte: 300, \$lte: 500 } })

```
usermanaged> db.transactions.find({"Transaction.Price":{"$gte:300,$lte:500"}})
[
  {
    _id: 2,
    Name: 'Ravi',
    Payment: { Total: 500 },
    Transaction: { Price: 350 }
  },
  {
    _id: 1,
    Name: 'Somu',
    Payment: { Total: 600 },
    Transaction: { Price: 400 }
  },
  {
    _id: 3,
    Name: 'Anusha',
    Payment: { Total: 500 },
    Transaction: { Price: 400 }
  }
]
```

d. Calculate the total transaction amount by adding up **Payment.Total** in all records.

```
db.transactions.aggregate([  
  {  
    $group: { _id: null, totalAmount: { $sum: "$Payment.Total" } }  
  }  
])
```

- **.aggregate()** is used for **aggregation operations**, which can perform **calculations, transformations, or grouping** on documents.
- **\$group** is used to group documents together based on a key.
- **_id** is the grouping key
- If **_id: null**, it means all documents are grouped into one single group.
- **\$sum** is a **MongoDB aggregation operator**.
- **"\$Payment.Total"** :- use the **value of this field** from each document."

```
usermanaged> db.transactions.aggregate([{$group:{_id:null,totalAmount:{$sum:"$Payment.Total"}}}])  
[ { _id: null, totalAmount: 2600 } ]
```

LAB PROGRAM 3

MERN STACK DEVELOPMENT BCI24457C

Lab program 3

- **a. Write a program to check request header for cookies.**
- **b. Write node.js program to print the car object properties, delete the second property and get length of the object.**

- HTTP headers are **key-value pairs** sent between a **client (browser)** and a **server**.
They carry **extra information (metadata)** about the request or response.
- You enter a URL in browser
- Browser sends an HTTP request
- Request includes:
 - Method (GET/POST)
 - Headers (like cookies, user-agent)
- Server receives the request
- Server processes it
- Server sends back a response
- Browser displays the result

3a. Write a program to check request header for cookies.

- It **creates a server**
- When a browser sends a request, it:
 - Reads all **HTTP headers**
- Checks if **cookies are present**
- Prints them in the **server console**
- Sends a simple message back to the browser
- **Browser → sends request → Server reads headers → checks cookies → prints result → sends response**

Steps for execution

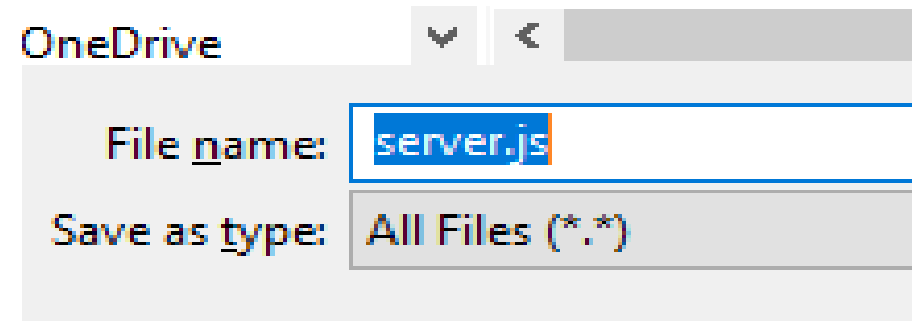
- Install **VisualStudio Code (VS CODE)**
- Install **Node.js**
- Go to: <https://nodejs.org>
- Verify installation of node js by :-
- Open terminal in VS Code or Command Prompt: **node -v**
- If versions appear → installed correctly

- **Open VS Code** Click File → Open Folder
- Create a folder lab3
- Click **New File** -> **Create JavaScript File** “server.js”
- Type Your Code and **save**
- Click: **Terminal** → **New Terminal**
- Run the Program

node server.js

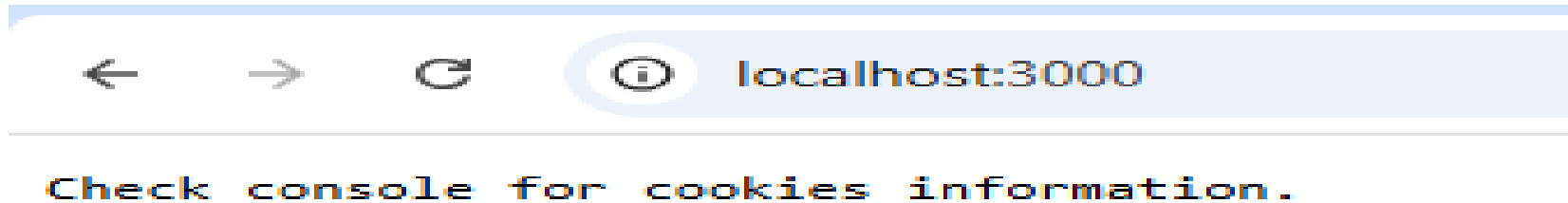
- You should see the Output:

Server running at <http://localhost:3000/>



```
C:\Users\Admin\lab3>node server.js
Server running at http://localhost:3000/
```

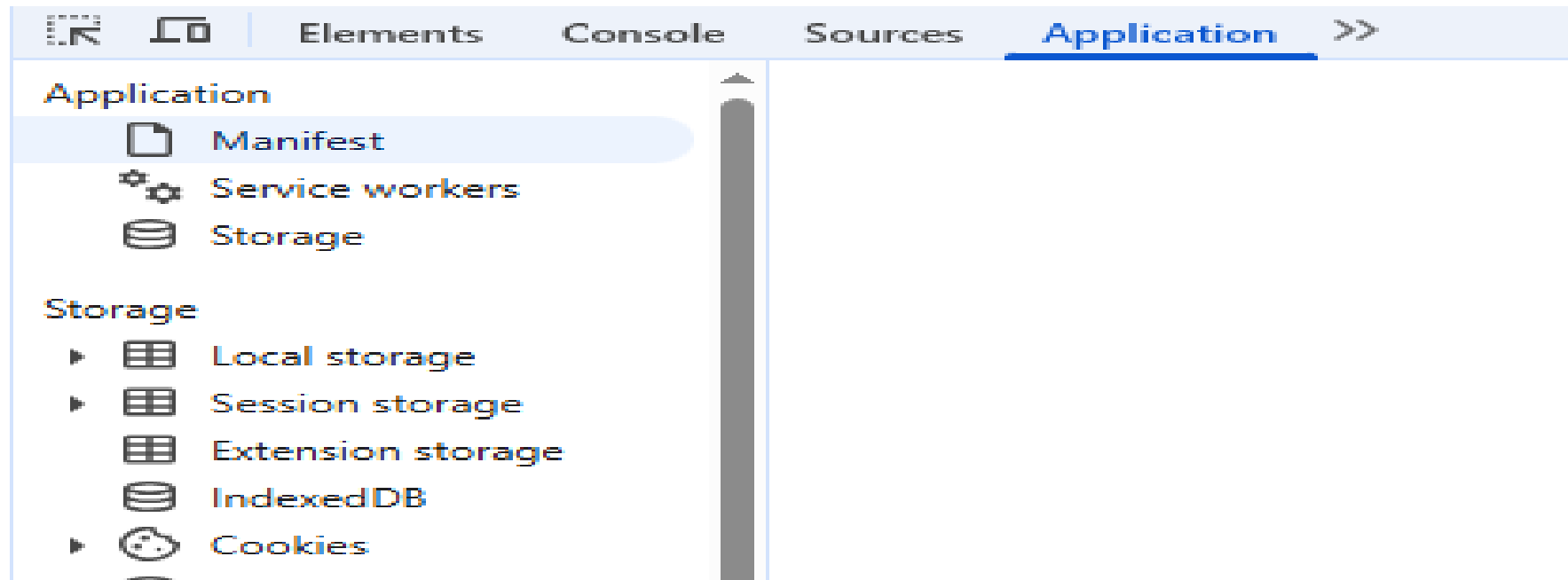
- **Open Chrome / Edge** : <http://localhost:3000>
- You should see the message "**Check console for cookies information.**" in the browser.



- Check VS Code Terminal to get the final output(you may get no cookies found)

```
referer: 'http://localhost:3000/',  
'accept-encoding': 'gzip, deflate, br, zstd',  
'accept-language': 'en-US,en;q=0.9'  
} .....  
No cookies found in the request header.  
|
```

- To add cookies
- In browser, Press F12
 - Using Browser Developer Tools:
 - Open the developer tools in your browser (usually by pressing F12).
 - Go to the "Application" or "Storage" tab.
 - In the "Cookies" section -> select localhost:3000.
 - Add Cookies by typing name and value (atleast 2 cookies)
 - Refresh the http://localhost:3000



- Now, look back at the terminal where you ran the Nodejs code. You'll see the request headers printed, and if there are cookies, you'll see cookies listed.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

  'accept-encoding': 'gzip, deflate, br, zstd',
  'accept-language': 'en-US,en;q=0.9',
  cookie: 'new=234'
} .....
Cookies: new=234
```

server.js

//imports the built-in **http module** in Node.js.

```
const http = require("http");
```

// Create a server

```
const server = http.createServer((req, res) =>
```

```
{
```

```
  const headers = req.headers; // Get request headers
```

```
  console.log(headers, ".....");
```

// Check if 'cookie' header exists

```
  if (headers.cookie)
```

```
  {
```

```
    console.log("Cookies:", headers.cookie);
```

```
  }
```

```
  else
```

```
  {
```

```
    console.log("No cookies found in the request header.");
```

```
  }
```

```
res.end("Check console for cookies information.");  
});  
// Listen on port 3000  
server.listen(3000, ( ) =>  
{  
  console.log("Server running at http://localhost:3000/");  
});
```

Output

- Open VSCode Terminal, Run the server:-
`node server.js`
- Server running at <http://localhost:3000/>
- Open browser <http://localhost:3000/>
- **"Check console for cookies information."**
- **Add Cookies (minimum 2) , Refresh the browser**
- **Go back to the terminal**
- **Cookies: name = value** (give the output that you have got)

3b Write node.js program to print the car object properties, delete the second property and get length of the object.

```
const car = {  
  make: 'Toyota',  
  model: 'Fortuner',  
  year: 2025,  
  color: 'black'  
};  
// Print the properties of the car object  
console.log('Original car object:');  
for (const property in car)  
{  
  console.log(` ${property}: ${car[property]} ` );  
}
```

// Delete the second property

```
const secondProperty = Object.keys(car)[1];
delete car[secondProperty];
console.log('\nCar object after deleting the second property:');
for (const property in car)
{
  console.log(` ${property}: ${car[property]} ` );
}
```

// Get the length of the object

```
const length = Object.keys(car).length;
console.log(` \nLength of the car object after deleting the second
property: ${length} ` );
```

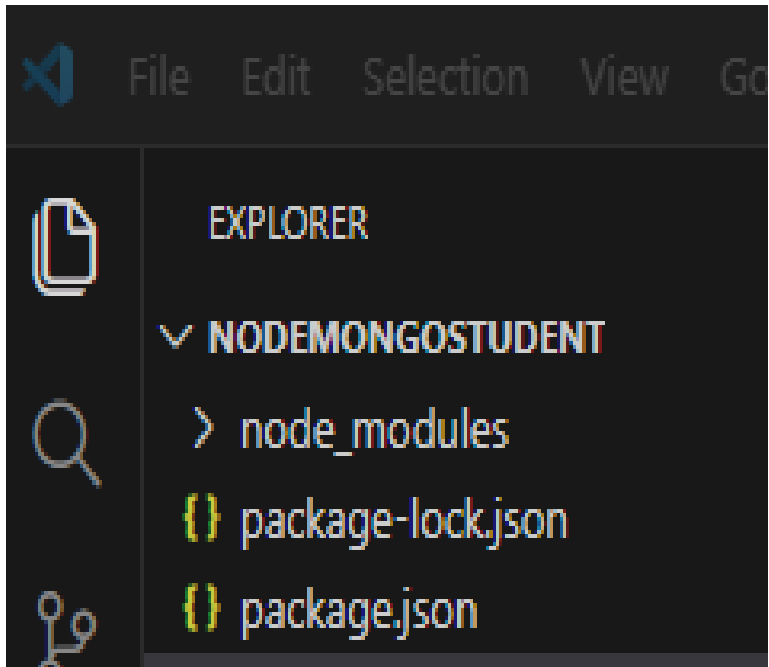
Lab program 4

MERN STACK DEVELOPMENT –BCI24457C

4 a. Read the data of a student containing usn, name, sem, year_of_admission from node js and store it in the mongodb.

- Open command prompt, type **mongod**
- **Open VS Code**
- Click **File → Open Folder**.
- Create/open a folder (example): NodeMongoStudent
- In VS Code:
- Terminal → New Terminal
- `npm init -y`
- `npm install mongodb`

- Now folder structure becomes



Create javascript file “Student.js”

```
const { MongoClient } = require('mongodb');  
const uri = 'mongodb://127.0.0.1:27017';  
async function main() {  
  const client = new MongoClient(uri);  
  try {  
    await client.connect();  
    const db = client.db('studentdb');  
    const students = db.collection('students');
```

```
const student = [  
  { usn: '4JK23AI001', name: 'Alice', sem: 4, year_of_admission: 2023 },  
  { usn: '4JK24AI005', name: 'Jane', sem: 2, year_of_admission: 2024 }  
];
```

```
const result = await students.insertMany(student);  
console.log(`Inserted ${result.insertedCount} students`);  
} catch(error) {  
  console.error("Error inserting data:", error);  
} finally {  
  await client.close();  
}  
}  
  
main().catch(console.error);
```

Output

- Save the file.
- In VS Code terminal run:
- `node Student.js`

```
PS C:\Users\Admin\NodeMongoStudent> node Student.js
● Inserted 2 students
○ PS C:\Users\Admin\NodeMongoStudent> |
```

Output contd.,

Verify using mongosh

- By using mongosh
- make sure mongod is running
- Open command prompt, type **mongosh**
- use **studentdb**
- **db.students.find()**

```
test> use studentdb
switched to db studentdb
studentdb> db.students.find()
[
  {
    _id: ObjectId('69b113430a9ca32431e4a7e2'),
    usn: '1GA23CS001',
    name: 'Alice',
    sem: 4,
    year_of_admission: 2023
  },
  {
    _id: ObjectId('69b113430a9ca32431e4a7e3'),
    usn: '1GA24CS005',
    name: 'Jane',
    sem: 2,
    year_of_admission: 2024
  }
]
```

Another way to check (optional)

Verify Using MongoDB Compass

- Verify Data in MongoDB either by mongosh prompt or MongoDB Compass
- Open MongoDB Compass from your system.
- You will see the connection screen.
- click **Connect**.
- After connecting, you will see a list of databases on the left.
- click on studentdb ; Open the Collection students; View Inserted Data

CONNECTIONS (1)

Search connections

localhost:27017

admin

college

config

local

studentdb

students

usermanaged

Type a query: { field: 'value' } or [Generate query](#)

ADD DATA

UPDATE

DELETE

EXPORT DATA

_id: ObjectId('69b113430a9ca32431e4a7e2')

usn : "1GA23CS001"

name : "Alice"

sem : 4

year_of_admission : 2023

_id: ObjectId('69b113430a9ca32431e4a7e3')

usn : "1GA24CS005"

name : "Jane"

sem : 2

year_of_admission : 2024

Program 4b:

For a partial name given in node js, search all the names from mongodb student documents created in Question(a).

```
const { MongoClient } = require('mongodb');
async function searchStudentsByName(partialName) {

const uri = 'mongodb://127.0.0.1:27017';
  const client = new MongoClient(uri);

  try {
    await client.connect();
    const database = client.db('studentdb');
    const students = database.collection('students');
    const query = { name: { $regex: partialName, $options: 'i' } };
    const cursor = students.find(query);
    const results = await cursor.toArray();
```

```
console.log(` Found ${results.length} students matching  
'${partialName}': ` );  
    results.forEach(student => console.log(student));  
} finally {  
    await client.close();  
}  
}  
searchStudentsByName('Al');
```

Output

- In VSCode Terminal

node search.js

```
PS C:\Users\Admin\NodeMongoStudent> node search.js
Found 1 students matching 'Al':
{
  _id: new ObjectId('69b113430a9ca32431e4a7e2'),
  usn: '1GA23CS001',
  name: 'Alice',
  sem: 4,
  year_of_admission: 2023
}
```

LAB PROGRAM 5

BCI24457C

5.Implement all CRUD operations on a File System using Node JS

- **Node.js** is a tool that lets to run **JavaScript outside the browser**.
- This is a menu-driven Node.js program that performs CRUD operations on a file using fs module and user input via readline.
- CRUD stands for:
 - **C – Create** → Create a new file
 - **R – Read** → Read the content of a file
 - **U – Update** → Modify the content of a file
 - **D – Delete** → Remove a file

- Steps:-
 - Create a folder lab5, and open that folder in Vscode
- OR
- Open VS code , New terminal
 - mkdir lab5
 - cd lab5
 - code . //used to open the folder
 - Create a file app.js

```
const fs = require('fs');  
const readline = require('readline');  
const path = require('path');  
const filePath = path.join(__dirname, 'sample.txt');
```

```
// Create interface for user input  
const rl = readline.createInterface({  
  input: process.stdin,  
  output: process.stdout  
});
```

// Menu function

```
function menu() {  
    console.log("\n--- FILE CRUD OPERATIONS ---");  
    console.log("1. Create File");  
    console.log("2. Read File");  
    console.log("3. Update File");  
    console.log("4. Delete File");  
    console.log("5. Exit");  
    rl.question("Enter your choice: ", (choice) => {  
        switch (choice) {  
            case '1':  
                createFile();  
                break;
```

```
case '2':  
    readFile();  
    break;  
case '3':  
    updateFile();  
    break;  
case '4':  
    deleteFile();  
    break;  
case '5':  
    console.log("Exiting...");  
    rl.close();  
    break;  
default:  
    console.log("Invalid choice!");  
    menu();  
}  
});  
}
```

// CREATE

```
function createFile() {  
  rl.question("Enter content to write: ", (content) => {  
    fs.writeFile(filePath, content, (err) => {  
      if (err) throw err;  
      console.log("File created successfully");  
      menu();  
    });  
  });  
}
```

// READ

```
function readFile() {  
  fs.readFile(filePath, 'utf8', (err, data) => {  
    if (err) {  
      console.log("File does not exist!");  
    } else {  
      console.log("File Content:\n" + data);  
    }  
    menu();  
  });  
}
```

// UPDATE

```
function updateFile() {  
  rl.question("Enter content to append: ", (content) => {  
    fs.appendFile(filePath, "\n" + content, (err) => {  
      if (err) throw err;  
      console.log("File updated successfully");  
      menu();  
    });  
  });  
}
```

// DELETE

```
function deleteFile() {  
  fs.unlink(filePath, (err) => {  
    if (err) {  
      console.log("File does not exist!");  
    } else {  
      console.log("File deleted successfully");  
    }  
    menu();  
  });  
}
```

// Start program

```
menu();
```

- Output
- In terminal, run
- node app.js
- **Creating file and reading file**

```
PS C:\Users\Admin\lab5> node app.js

--- FILE CRUD OPERATIONS ---
1. Create File
2. Read File
3. Update File
4. Delete File
5. Exit
Enter your choice: 1
Enter content to write: hello world
File created successfully

--- FILE CRUD OPERATIONS ---
1. Create File
2. Read File
3. Update File
4. Delete File
5. Exit
Enter your choice: 2
File Content:
hello world
```

- Updating the file and reading the file

```
--- FILE CRUD OPERATIONS ---
```

```
1. Create File
```

```
2. Read File
```

```
3. Update File
```

```
4. Delete File
```

```
5. Exit
```

```
Enter your choice: 3
```

```
Enter content to append: good morning all
```

```
File updated successfully
```

```
--- FILE CRUD OPERATIONS ---
```

```
1. Create File
```

```
2. Read File
```

```
3. Update File
```

```
4. Delete File
```

```
5. Exit
```

```
Enter your choice: 2
```

```
File Content:
```

```
hello world
```

```
good morning all
```

- **Deleting the file and checking whether it got deleted**

```
--- FILE CRUD OPERATIONS ---
```

```
1. Create File
```

```
2. Read File
```

```
3. Update File
```

```
4. Delete File
```

```
5. Exit
```

```
Enter your choice: 4
```

```
File deleted successfully
```

```
--- FILE CRUD OPERATIONS ---
```

```
1. Create File
```

```
2. Read File
```

```
3. Update File
```

```
4. Delete File
```

```
5. Exit
```

```
Enter your choice: 2
```

```
File does not exist!
```

LAB PROGRAM 6

BCI24457C

6. Develop the application that sends fruit name and price data from client side to Node.js server using Ajax.

- This application collects fruit name and price from a web form and sends the data to a Node.js server using AJAX (fetch API).
- The server receives the data using Express and processes it.\
- The server then sends a response back to the client, which is displayed without reloading the webpage.

- **Steps:-**
- **Install node js**
- Create project folder in VS Code – fruit-ajax
- Terminal → New Terminal

npm init -y

- Install Express and CORS

npm install express cors

Create file: server.js

```
const express = require("express");
const cors = require("cors");

const app = express();
const PORT = 5000;

app.use(express.json());
app.use(cors());

app.post("/fruit", (req, res) => {
  const { name, price } = req.body;

  console.log("Fruit Name:", name);
  console.log("Price:", price);

  res.send("Data received successfully");
});

app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});
```

Create file: index.html

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>Fruit Form</title>
```

```
</head>
```

```
<body>
```

```
<h2>Fruit Data</h2>
```

```
<form id="fruitForm">
```

Fruit Name:

```
<input type="text" id="name"><br><br>
```

Price:

```
<input type="text" id="price"><br><br>
```

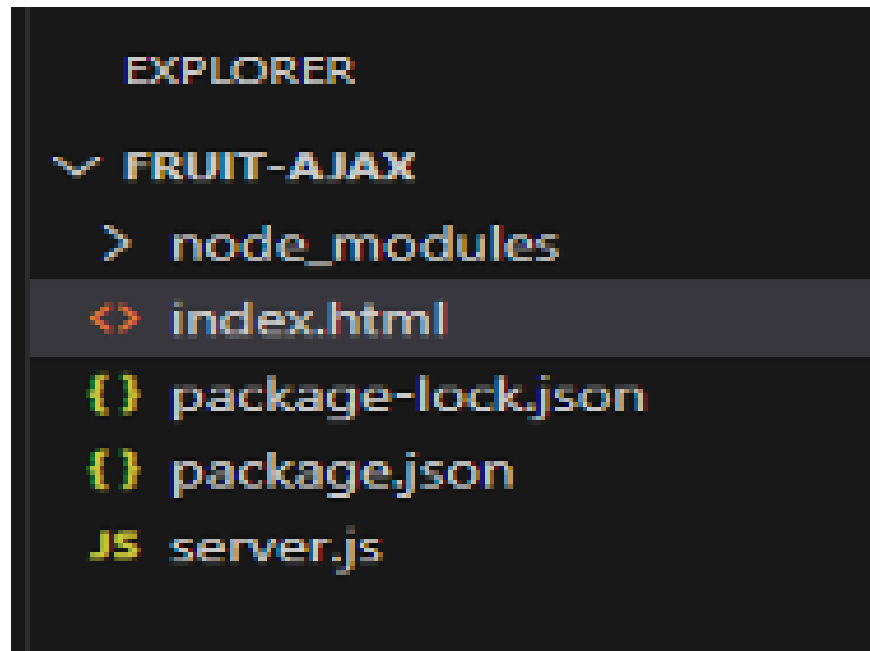
```
<button type="submit">Submit</button>
```

```
</form>
```

```
<script>
document.getElementById("fruitForm").addEventListener("submit", function(e) {
e.preventDefault();
let name = document.getElementById("name").value;
let price = document.getElementById("price").value;
fetch("http://localhost:5000/fruit", {
method:"POST",
headers: {
"Content-Type":"application/json"
},
body:JSON.stringify({
name:name,
price:price
})
})
.then(res=>res.text())
.then(data=>alert(data));
});
</script>
</body>
</html>
```

Folder Structure

Your folder should look like:



- **Output**
- **Run the Server**
- **node server.js**
- Output:
- Server running on http://localhost:5000

Run the Client

- Open index.html
- Right click
- Open with Browser *//record your output*
- Go back to terminal and check the output *//record your output*

LAB PROGRAM 7

BCI24457C

7. Develop an authentication mechanism with email_id and password using HTML and Express JS (POST method)

Steps: -

- In VS Code: Create LAB7 folder
- Click Terminal -> New Terminal

npm init -y

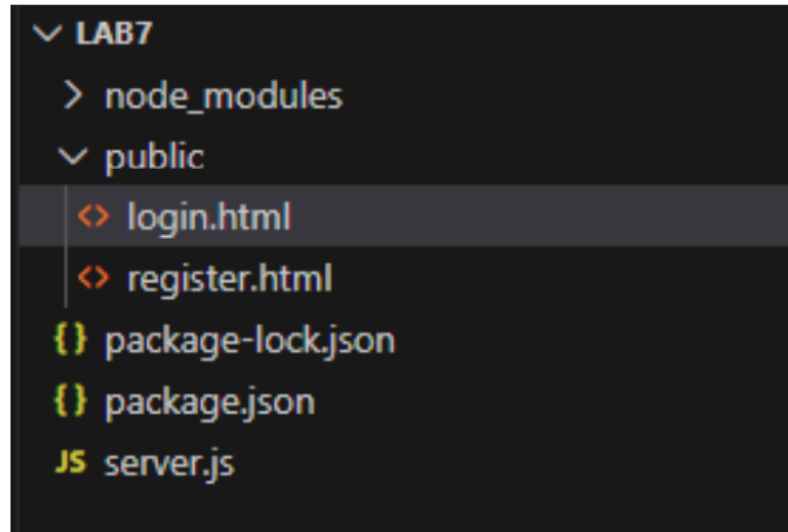
This creates package.json

npm install express bcrypt

- wait for installation to get over In left panel (Explorer),
- Click New File, server.js (paste the code and save)
- mkdir public

Inside public create:

- register.html (type the code and save)
- login.html (type the code and save)



Server.js

```
const express = require('express');
const bcrypt = require('bcrypt');
const app = express();
const PORT = 3000;

// Middleware for form data
app.use(express.urlencoded({ extended: true }));

// Serve HTML files
app.use(express.static('public'));

// Temporary storage
let users = [];

// Register
app.post('/register', async (req, res) => {
  const { email, password } = req.body;
  const user = users.find(u => u.email === email);
  if (user) {
    return res.send("User already exists");
  }
});
```

```
// Hash password
const hashedPassword = await bcrypt.hash(password, 10);
users.push({ email, password: hashedPassword });
res.send("Registration successful");
});
// Login
app.post('/login', async (req, res) => {
  const { email, password } = req.body;
  const user = users.find(u => u.email === email);
  if (!user) {
    return res.send("User not found");
  }

  // Compare password
  const match = await bcrypt.compare(password, user.password);
  if (match) {
    res.send("Login successful");
  } else {
    res.send("Wrong password");
  }
});
app.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
});
```

Register.html

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>Register</title>
```

```
  <style>
```

```
    body {
```

```
      background-color: #e3f2fd;
```

```
      font-family: Arial;
```

```
    }
```

```
    form {
```

```
      background-color: rgb(223, 233, 190);
```

```
      width: 350px;
```

```
      margin: auto;
```

```
      margin-top: 100px;
```

```
      padding: 20px;
```

```
    }
```

```
    input {
```

```
      width: 180px; /* same size for all boxes */
```

```
    }
```

```
  </style>
```

```
</head>
```

```
<body>
  <form action="/register" method="POST">
    <h2 align="center">Register</h2>
    <table align="center">
      <tr>
        <td>Email :</td>
        <td><input type="email" name="email" required></td>
      </tr>

      <tr>
        <td>Password :</td>
        <td><input type="password" name="password" required></td>
      </tr>
    </table>
    <br>
    <center>
      <button type="submit">Register</button>
    </center>
    <p align="center"><a href="login.html">Go to Login</a></p>
  </form>
```

```
</body>
</html>
```

Login.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Login</title>
  <style>
    body {
      background-color: #e3f2fd;
      font-family: Arial;
    }

    form {
      background-color: rgb(228, 191, 191);
      width: 350px;
      margin: auto;
      margin-top: 100px;
      padding: 20px;
    }

    input {
      width: 180px;
    }
  </style>
</head>
```

```
<body>
  <form action="/login" method="POST">
    <h2 align="center">Login</h2>

    <table align="center">
      <tr>
        <td>Email :</td>
        <td><input type="email" name="email" required></td>
      </tr>

      <tr>
        <td>Password :</td>
        <td><input type="password" name="password" required></td>
      </tr>
    </table>

    <br>
    <center>
      <button type="submit">Login</button>
    </center>

    <p align="center"><a href="register.html">Go to Register</a></p>
  </form>
</body>
</html>
```

8. Develop two routes: find_prime_100 and find_cube_100 which prints prime numbers less than 100 and cubes less than 100 using Express JS routing mechanism.

```
const express = require('express');
```

```
const app = express();
```

```
function isPrime(num) {  
  if (num <= 1) return false;  
  for (let i = 2; i < Math.sqrt(num); i++) {  
    if (num % i === 0) return false;  
  }  
  return true;  
}
```

```
app.get('/find_prime_100', (req, res) => {  
  const primes = [];  
  for (let i = 2; i < 100; i++) {  
    if (isPrime(i)) {  
      primes.push(i);  
    }  
  }  
  res.json({ message: 'Prime numbers less than 100:', primes });  
});
```

```
app.get('/find_cube_100', (req, res) => {  
  const cubes = [];  
  for (let i = 1; i < 10; i++) {  
    cubes.push(i * i * i);  
  }  
  res.json({ message: 'Cubes less than 100:', cubes });  
});
```

```
});
```

```
app.listen(3000, () => console.log('Server listening on port 3000'));
```

Explanation:

1. We import `express` to create an Express application.
2. We define a function `isPrime` that checks if a number is prime.
3. We define two routes:
 - `/find_prime_100`: This route uses a GET method. It iterates through numbers from 2 to 99 (less than 100) and checks if they are prime using the `isPrime` function. If prime, it adds the number to a `primes` array. Finally, it sends a JSON response with a message and the list of prime numbers.
 - `/find_cube_100`: This route is also a GET method. It iterates through numbers from 1 to 9 (less than 10) and calculates the cube by multiplying the number by itself three times. It adds the cube to a `cubes` array. Finally, it sends a JSON response with a message and the list of cubes.
4. We listen on port 3000 for incoming requests.

This code defines two routes that handle GET requests and respond with JSON data containing the prime numbers and cubes less than 100. You can access these routes by sending GET requests to `http://localhost:3000/find_prime_100` and `http://localhost:3000/find_cube_100` in your browser or using tools like Postman.

Run

1. `npm install express`
2. `node filename.js`

LAB 9

MERN STACK DEVELOPMENT

BCI24457C

9. Develop a React code to build a simple search filter functionality to display a filtered list based on the search query entered by the user.

- Develop a simple **Search Filter** application using React.
- A list of items is already stored in data.json.
- The user types something into a search box.
- React filters the list and shows only matching items.
- If nothing matches, it displays “No items found”.

React

- React is a **JavaScript library** used to build **interactive user interfaces (UI)**.
- It is mainly used for:
 - Websites
 - Single Page Applications
 - Dynamic web pages

1. Component

A component is a reusable part of the UI.

2. State

State stores changing data in React.

3. Props

Props are used to pass data from one component to another.

4. JSX (JavaScript XML)

JSX allows writing HTML inside JavaScript.

Steps:-

- Install node js
- Open VS Code terminal and run:
npx create-react-app search-filter
cd search-filter
code .
- Create Required Files
- **Inside src folder,**
- create:
 SearchFilter.js
 data.json

EXPLORER

▼ SEARCH-FILTER

> node_modules

> public

▼ src

App.css

JS App.js

JS App.test.js

{ } data.json

index.css

JS index.js

📁 logo.svg

JS reportWebVitals.js

JS SearchFilter.js

JS setupTests.js

📄 .gitignore

{ } package-lock.json

{ } package.json

📖 README.md

Open index.js, Replace with:

```
import React from 'react';  
import ReactDOM from 'react-dom/client';  
import App from './App';
```

```
const root = ReactDOM.createRoot(document.getElementById('root'));  
root.render(  
  <React.StrictMode>  
    <App />  
  </React.StrictMode>  
);
```

- *React is needed to create components and JSX.*
- *ReactDOM connects React with the browser DOM.*
- *Imports the App component from App.js.*

- *<App />*
- *Displays the App component.*
- *<React.StrictMode>*
- *Used for checking warnings and errors during development.*

- Browser loads page
- index.js runs
- App component loaded
- Displayed inside root div

Open app.js and replace with :-

App.js

```
import React from "react";  
import SearchFilter from "./SearchFilter";
```

```
function App() {  
  return (  
    <div>  
      <h1>Simple Search Filter</h1>  
      <SearchFilter />  
    </div>  
  );  
}  
export default App;
```

<SearchFilter />

Calls the SearchFilter component.

This component contains:

- search box
- filtering logic
- list display

THE FLOW IS :-

- App Component -> Shows Heading-> Loads SearchFilter Component

SearchFilter.js

```
import React, { useState } from "react";  
import data from "./data.json";
```

```
function SearchFilter() {  
  const [query, setQuery] = useState("");  
  const filteredItems = data.filter((item) =>  
    item.name.toLowerCase().includes(query.toLowerCase())  
  );  
  return (  
    <div>  
      <input  
        type="text"  
        placeholder="Search item..."  
        value={query}  
        onChange={(e) => setQuery(e.target.value)}  
      />  
    </div>  
  );  
}
```

query

Stores current search text.

setQuery

Function used to update query.

useState(" ")

Initial value is empty string.

Runs whenever user types.

e

Represents event.

e.target.value

Gets typed text.

setQuery()

Updates query state.

EX:- If user types **car** then,

setQuery("car")

```

<ul>
  {filteredItems.map((item, index) => (
    <li key={index}>
      {item.name} - {item.category}
    </li>
  )))}

  {filteredItems.length === 0 && <p>No items found</p>}}
</ul>
</div>

);
}

export default SearchFilter;

```

map() is a JavaScript array method. It processes each array element one by one and returns new elements.

For ex:- filteredItems contains

```

[
  { name: "Banana", category: "Fruit" },
  { name: "Beans", category: "Vegetable" }
]

```

In 1st iteration:-

```

item = { name: "Banana", category: "Fruit" }

```

index = 0

React creates:

```

<li key = {0}>
  Banana - Fruit
</li>

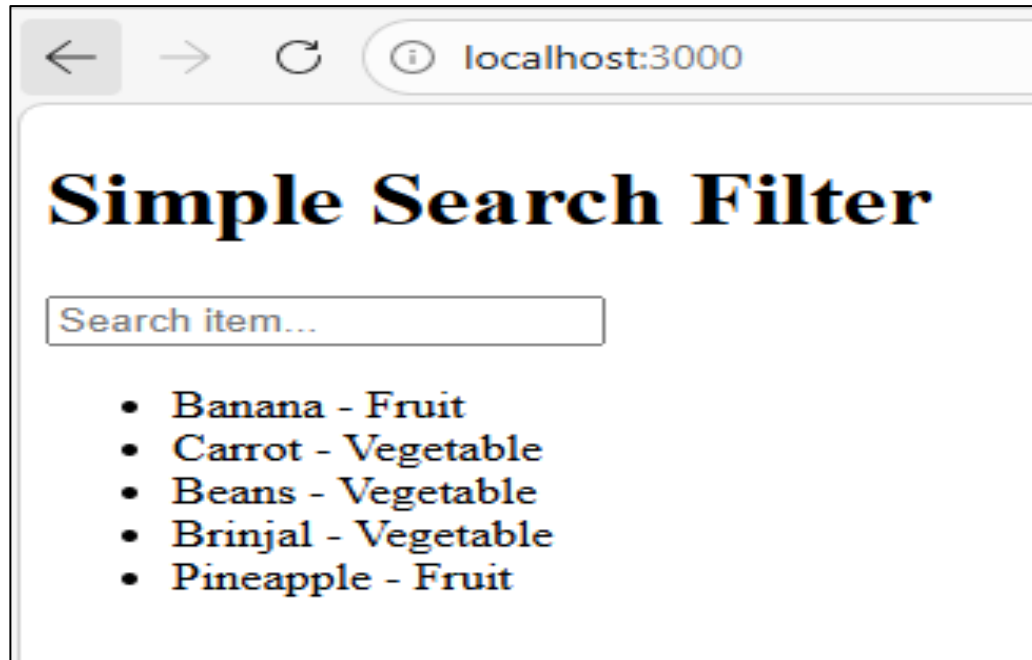
```

Complete workflow

- User types in search box
- onChange event occurs
- setQuery updates state
- React re-renders component
- filter() checks matching items
- Matching items displayed

Output

- Run the React App
- Open Terminal
- **npm start**
- Browser will open automatically:
- **http://localhost:3000**



LAB PROGRAM 10

BCI24457C

10. Develop a React code to collect data from rest API.

- This is a React application that fetches data from a REST API and displays it on a webpage.
- The app requests data from a server (API)
- Receives the data
- Displays it in the browser

REST API = Representational State Transfer Application Programming Interface

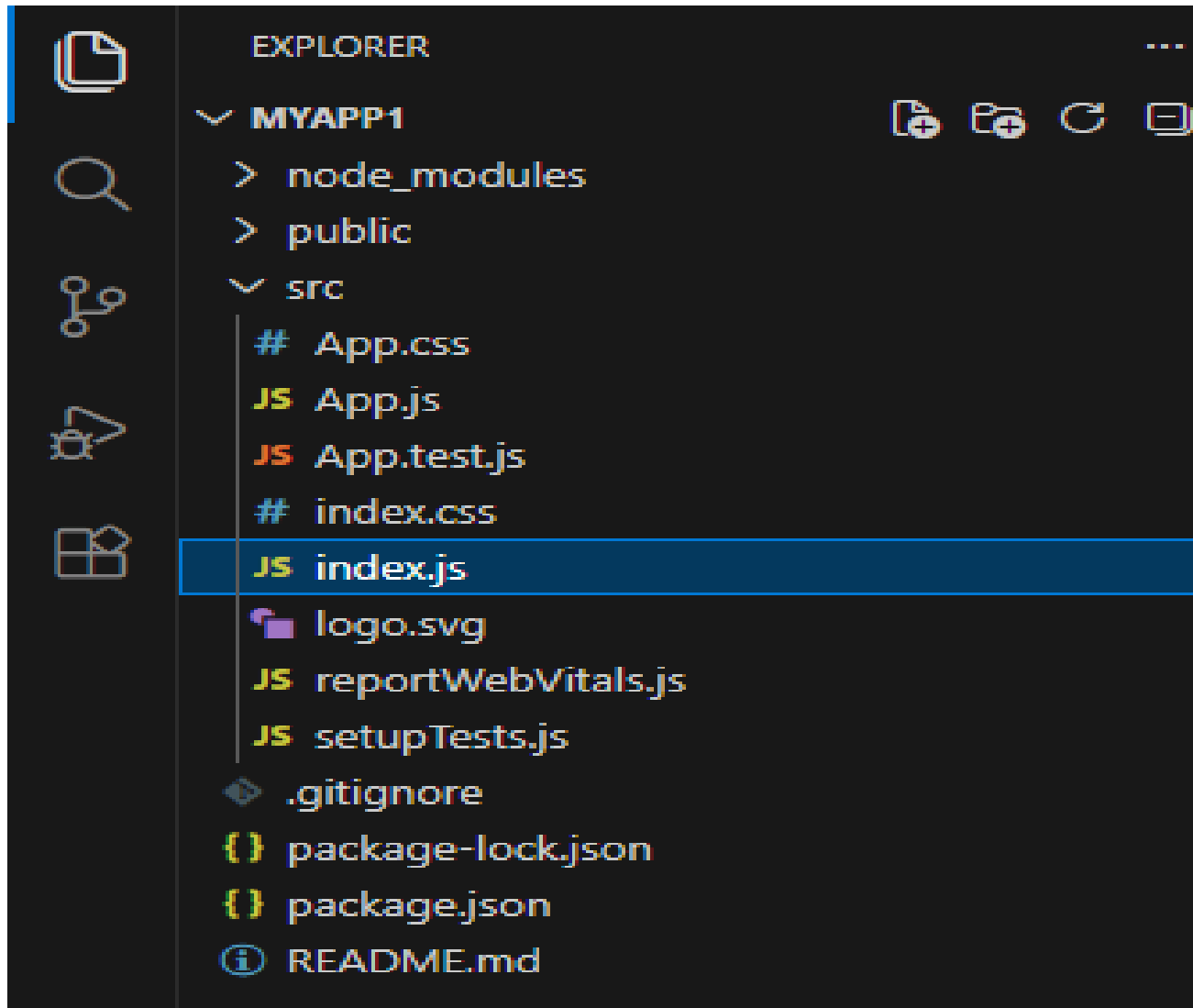
- It is a method that allows:
- Frontend - Backend communication
- Applications - Servers communication
- using the internet.

IN DETAIL

- REST API is an online service that provides data.
- The program uses the API:
<https://dummy.restapiexample.com/api/v1/employees>
- This API returns employee details in JSON format.
- The data is fetched using **Axios**, which is a library used to make HTTP requests.
- The employee names are then displayed on the screen.

Steps:-

- Install node.js
- Open VS Code
- Click Terminal → New Terminal
- **Run the command**
npm create-react-app myapp
cd myapp
code .
- code . opens project in VS Code
npm install axios



Update src/index.js

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import './index.css';
import App from './App';

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

Update src/App.js

```
import React, { useState, useEffect } from "react";  
import axios from "axios";
```

```
function DataCollector() {  
  const [data, setData] = useState([]);
```

```
  useEffect(() => {  
    fetchData();  
  }, []);
```

- **useState** :- Used to store data/state.
- **useEffect** :- Used to perform actions when the component loads.
- **Axios** is used to send HTTP requests to APIs.

useEffect :- Runs automatically when the component loads for the first time.

*Empty dependency array means:
"Run only once when the page loads."*

```
const fetchData = async () => {  
  try {  
    const response = await axios.get(  
      "https://dummy.restapiexample.com/api/v1/employees"  
    );  
    setData(response.data.data);  
  } catch (error) {  
    console.error("Error fetching data:", error);  
  }  
};
```

*The API response contains
nested JSON.*

response.data :- full JSON
object.

response.data.data :- actual
employee array.

Returns the UI that appears on webpage.

```
return (  
  <div>  
    <h2>Data Collection</h2>  
    <button onClick={fetchData}>Fetch Data</button>  
    <ul>  
      {data.map((item, index) => (  
        <li key={index}>{item.employee_name}</li>  
      ))}  
    </ul>  
  </div>  
);  
}  
export default DataCollector;
```

map() Function

Loops through each employee object in data.

- *item* → *current employee object.*
- *index* → *position number.*

Output

- In VS Code terminal type:-
npm start
- Browser opens automatically
URL: <http://localhost:3000>

